Fundamentals of Data Science

Assignment 4

Name: Praptee Dewan

Student ID: 25068590

Module Code: UFCFK1-15-0

Instructor: Kaushal Rimal

Submission Date: 17 April 2026

Github link: https://github.com/dprapti25-coder/FODS-Submissions.git

1) **Numpy Array Operations**

Program Description:

This program creates a NumPy array and performs basic statistical operations such as sum, mean, maximum, and minimum.

Code:

```
'''
NumPy array operations
Creates a NumPy array and finds sum, mean, max, and min values
'''

import numpy as np

arr = np.array([10, 20, 30, 40, 50]) #1D array created

print("Array:", arr)

print("Sum:", np.sum(arr))
print("Mean:", np.mean(arr))
print("Max:", np.max(arr))
print("Min:", np.min(arr))
```

Output:

```
Array: [10 20 30 40 50]
Sum: 150
Mean: 30.0
Max: 50
Min: 10
```

Explanation:

The program uses NumPy functions like np.sum(), np.mean(), np.max(), and np.min() to compute values from an array.

2) **List sorting and slicing**

Program Description:

This program takes a list of numbers, sorts them, and performs slicing operations to extract specific index ranges.

Code:

```
'''
List sorting and slicing
Takes 12 numbers, sorts them, and performs slicing
'''

numbers = input("Enter at least 12 numbers: ").split() #splits the string into
a list of strings
numbers = [int(x) for x in numbers] #string elements are converted to integer
for sorting

numbers.sort()

print("Sorted list:", numbers)

print("3-6:", numbers[3:7]) #slicing follows the [start::stop] format
print("6-9:", numbers[6:10])
print("4-10:", numbers[4:11])
```

Output:

```
Enter at least 12 numbers: 1 2 3 4 5 6 7 8 9 10 11 12
Sorted list: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
3-6: [4, 5, 6, 7]
6-9: [7, 8, 9, 10]
4-10: [5, 6, 7, 8, 9, 10, 11]
PS C:\Users\praka\OneDrive\Desktop\TBC\FODS\TAKE HOME A
ISSION>
```

Explanation:

The list is sorted using .sort() and slicing is performed using Python indexing [start:end].

3) **Numpy Array Reshape**

Program Description:

This program generates a random NumPy array, sorts it, and reshapes it into a matrix.

Code:

```
'''
NumPy reshape
Creates random array, sorts it, and reshapes into matrix
'''

import numpy as np

arr = np.random.randint(1, 100, 12)
#this generates an array with 12 random integers between 1-100

print("Original:", arr)

arr.sort()

print("Sorted:", arr)

matrix = arr.reshape(3, 4)

print("Reshaped (3x4):")
print(matrix)
```

Output:

```
Original: [ 8 94 58 55  3 83 84 97 13 66 97 24]
Sorted: [ 3  8 13 24 55 58 66 83 84 94 97 97]
Reshaped (3x4):
[[ 3  8 13 24]
 [55 58 66 83]
 [84 94 97 97]]
```

Explanation:

The array is reshaped using .reshape(rows, columns) after ensuring total elements match matrix size.

4) MATRIX OPERATIONS USING NUMPY

Program Description:

This program takes two matrices and performs addition, subtraction, and multiplication after validating their dimensions.

Code:

```
'''
Matrix operations using NumPy
Takes two matrices and performs +, -, * operations
'''
```

```python
import numpy as np

r = int(input("Rows: "))
c = int(input("Columns: "))

print("Matrix 1:")
m1 = np.array([[int(input()) for j in range(c)] for i in range(r)])
#Matrix 1 is created using for loops
#inner loop handles columns and the outer handles rows

print("Matrix 2:")
m2 = np.array([[int(input()) for j in range(c)] for i in range(r)])

#in numpy '*' does not function as a dot product instead it performs element-
wise multiplication
print("Addition:\n", m1 + m2)
print("Subtraction:\n", m1 - m2)
print("Multiplication:\n", m1 * m2)
```

Output:

```
Rows: 1
Columns: 3
Matrix 1:
2
4
6
Matrix 2:
5
0
8
Addition:
 [[ 7  4 14]]
Subtraction:
 [[-3  4 -2]]
Multiplication:
 [[10  0 48]]
```

Explanation:

NumPy arrays are used for matrix operations. If dimensions do not match, an exception is raised.

5) **Pandas and Matplotlib**

Program Description:

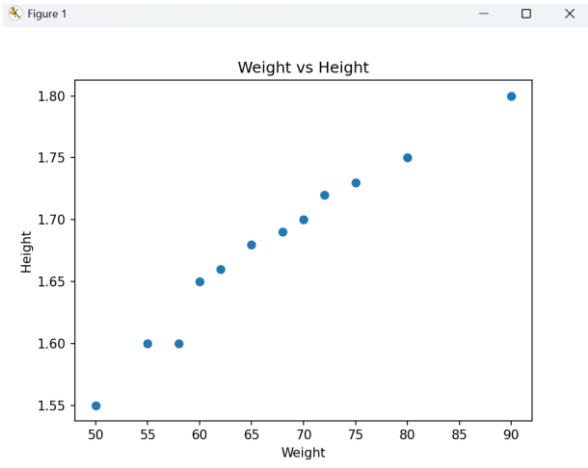
This program reads health data from a CSV file and generates scatter plots for different variable relationships.

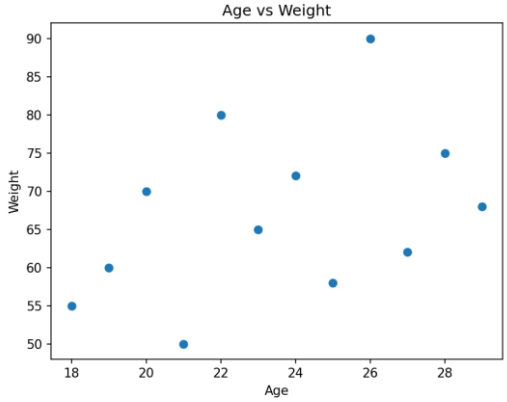
Code:

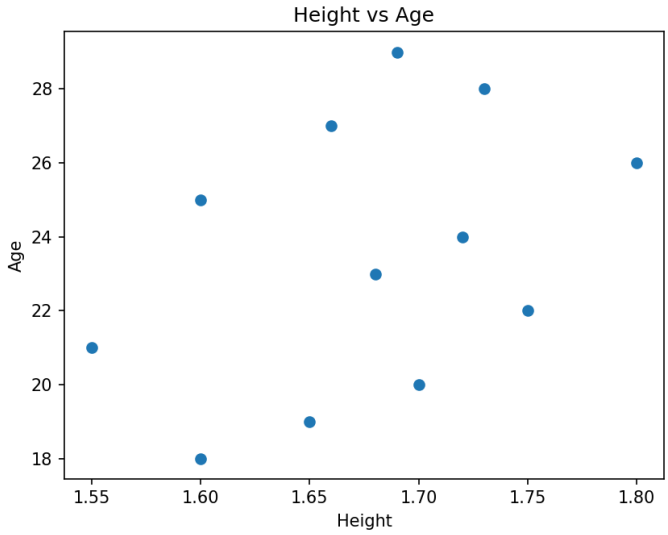
```
'''
Health Data Visualization using Pandas and Matplotlib
Reads health_data.csv and creates scatter plots for different relationships
'''

import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("health_data.csv")

# Convert gender to numeric for plotting (M=1, F=0)
df["gender_num"] = df["gender"].map({"M": 1, "F": 0})

# 1. Weight vs Height
plt.scatter(df["weight"], df["height"])
plt.title("Weight vs Height")
plt.xlabel("Weight")
plt.ylabel("Height")
plt.show()

# 2. Age vs Weight
plt.scatter(df["age"], df["weight"])
plt.title("Age vs Weight")
plt.xlabel("Age")
plt.ylabel("Weight")
plt.show()
```

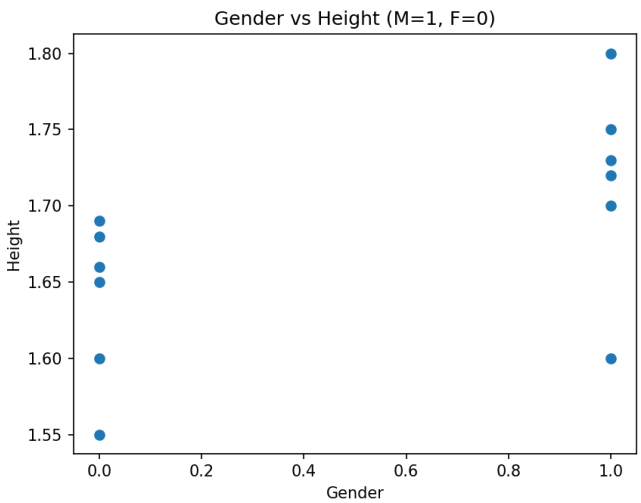
```
# 3. Height vs Age
plt.scatter(df["height"], df["age"])
plt.title("Height vs Age")
plt.xlabel("Height")
plt.ylabel("Age")
plt.show()

# 4. Gender vs Height
plt.scatter(df["gender_num"], df["height"])
plt.title("Gender vs Height (M=1, F=0)")
plt.xlabel("Gender")
plt.ylabel("Height")
plt.show()

# 5. Gender vs Weight
plt.scatter(df["gender_num"], df["weight"])
plt.title("Gender vs Weight (M=1, F=0)")
plt.xlabel("Gender")
plt.ylabel("Weight")
plt.show()
```

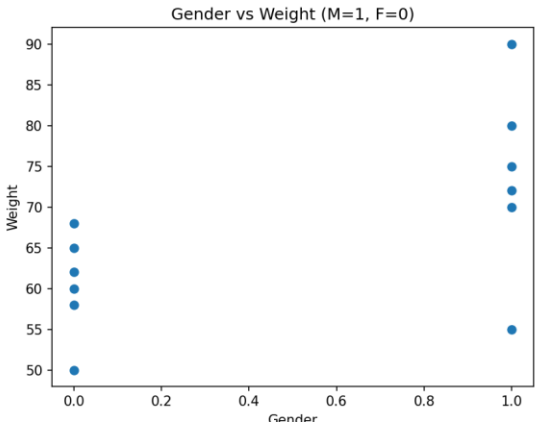
Output (Graphs):




Age vs Weight
Weight
Age
90
85
80
75
70
65
60
55
50
18
20
22
24
26
28


Height vs Age
Age
Height
28
26
24
22
20
18
1.55
1.60
1.65
1.70
1.75
1.80


Gender vs Height (M=1, F=0)
Height
Gender
1.80
1.75
1.70
1.65
1.60
1.55
0.0
0.2
0.4
0.6
0.8
1.0



Explanation:

Pandas reads the dataset and Matplotlib is used to visualize relationships between variables like age, weight, height, and gender.

6) **Health status and BMI**

Program Description:

This program calculates BMI and assigns a health status category based on BMI values.

Code:

```
'''
BMI Calculation and Health Status Classification
Adds BMI and Health_Status columns to dataset using conditions.
'''

import pandas as pd

# Load dataset
df = pd.read_csv("health_data.csv")

# Calculate BMI
df["BMI"] = df["weight"] / (df["height"] ** 2)

# Function to classify health status
def get_health_status(bmi):
    if bmi < 18.5:
        return "Underweight"
    elif 18.5 <= bmi <= 24.9:
        return "Healthy range"
    elif 25 <= bmi <= 29.9:
        return "Overweight risk"
    elif 30 <= bmi <= 34.9:
        return "High risk"
    else:
        return "Critical condition"

# Apply function to create new column
df["Health_Status"] = df["BMI"].apply(get_health_status)

# Display updated dataset
print(df)

# Save new file
df.to_csv("updated_health_data.csv", index=False)

print("\nUpdated file saved as updated_health_data.csv")
```

Output:

```
    age  weight  ...        BMI    Health_Status
0    18      55  ...  21.484375    Healthy range
1    19      60  ...  22.038567    Healthy range
2    20      70  ...  24.221453    Healthy range
3    21      50  ...  20.811655    Healthy range
4    22      80  ...  26.122449  Overweight risk
5    23      65  ...  23.030045    Healthy range
6    24      72  ...  24.337480    Healthy range
7    25      58  ...  22.656250    Healthy range
8    26      90  ...  27.777778  Overweight risk
9    27      62  ...  22.499637    Healthy range
10   28      75  ...  25.059307  Overweight risk
11   29      68  ...  23.808690    Healthy range

[12 rows x 6 columns]

Updated file saved as updated_health_data.csv
```

Explanation:

A new BMI column is created using weight/height formula, and .apply() is used to assign health categories.